

Тестовое задание

Mini Transcribe Bot — мини-сервис «аудио → транскрипция → LLM-анализ»

Для кого: ученики НИШ **Время:** 4–8 часов в течение 5 дней

Язык: Python 3.11+ (или TypeScript/Node — на выбор) **Уровень:** middle-junior



Привет, я Юрий

Предприниматель · Основатель LoyaBonus.app

Резидент Oskemen Hub

Я построил **LoyaBonus.app** — платформу лояльности для бизнеса, и развиваю её как продукт. Сейчас мы открываем **стажировку для учеников НИШ** с возможностью перехода в штат после её успешного прохождения. Это не «принеси кофе и посмотри как взрослые работают» — это реальная работа над реальным продуктом с первого дня.

ЧЕМ БУДЕШЬ ЗАНИМАТЬСЯ

- писать код и фичи для нашего продукта вместе с командой;
- делать интеграции с внешними API (платежи, мессенджеры, аналитика, AI-провайдеры);
- собирать небольшие приложения и инструменты под задачи бизнеса;
- править и улучшать визуальную часть (UI/UX), доводить до «не стыдно показать»;
- **и всё это — с активным использованием ИИ** (Claude, Cursor, ChatGPT, Copilot). Вайбкодинг — наш стандартный режим работы, а не «секретный лайфхак».

ПОЧЕМУ ЭТО ИНТЕРЕСНО

- современный стек и AI-инструменты, которые сейчас определяют рынок;
- учишься не «прохождением курсов», а решением задач, за которыми стоят настоящие пользователи;
- видишь, как продукт устроен изнутри: код → деплой → клиенты → деньги;
- при хорошем результате — оффер в штат и рост вместе с компанией.

Зачем это тестовое. Мы хотим увидеть, как ты собираешь маленький, но **живой** продукт: как работаешь с AI-ассистентом, как тестируешь чужие API, как обращаешься с секретами и ошибками. Финальный код будет небольшим (≤ 500 строк), но за ним должно быть видно мышление инженера, а не «накидал и сдал».

1. Что нужно сделать

Сервис, который принимает аудиофайл от авторизованного пользователя, отправляет его в API транскрипции, затем — в LLM для структурированного анализа, и возвращает результат. Можно сделать как CLI + HTTP API, либо только HTTP API (на выбор).

Поток работы

1. Пользователь регистрируется / логинится (email + пароль, JWT или session — на выбор).
2. Загружает аудиофайл (`.mp3` / `.wav` , до 10 минут).
3. Сервис проверяет лимит (см. ниже), отправляет файл в **API транскрипции**, получает текст.
4. Отправляет текст в **LLM API**, который возвращает **строго структурированный JSON**:

```
{
  "summary": "string, 2-4 предложения",
  "topics": ["string", ...],
  "sentiment": "positive" | "neutral" | "negative",
  "action_items": ["string", ...]  // может быть пустым
}
```

5. Результат (транскрипт + JSON-отчёт) сохраняется и доступен пользователю по его ID.

Авторизация и лимиты

- Регистрация и вход обязательны — без авторизации никаких операций.
- Подписки/тарифов нет. У каждого пользователя (tenant) — фиксированный **бесплатный лимит: 30 минут аудио суммарно**.
- Длительность считается до запуска транскрипции (через `ffprobe` или эквивалент). Если файл превышает остаток лимита — отказ с понятным сообщением.
- Использованные минуты считаются на стороне сервера и не сбрасываются.

- Эндпоинт `GET /me/usage` — возвращает использовано/осталось.

Минимальный набор эндпоинтов (если HTTP API)

```
POST /auth/register      { email, password }
POST /auth/login         { email, password } → token
POST /jobs               multipart: audio file → job_id
GET  /jobs/:id           → status, transcript, report
GET  /me/usage           → { used_minutes, limit_minutes: 30 }
```

2. Какие API использовать

Мы намеренно не указываем конкретного провайдера. Часть задания — выбрать самому и обосновать в README. Подойдут любые с бесплатным тиром:

Транскрипция	LLM
Deepgram (free \$200), AssemblyAI (free 100ч), Groq Whisper (free), OpenAI Whisper	OpenRouter (бесплатные модели DeepSeek/Llama), Groq, Google AI Studio (Gemini Flash free)

Важно: ключи API **не должны** попасть в репозиторий. Конфигурация — через `.env` и `os.environ`. В репозитории — только `.env.example`.

3. Обязательные требования

- **README.md:** как установить, как запустить, какие провайдеры выбрал и почему, какие были подводные камни.
- **Зависимости:** зафиксированы (`requirements.txt` / `pyproject.toml` / `package.json` с lock-файлом).
- **Хранение пользователей:** любая БД (SQLite / Postgres). Пароли — только хешированные (bcrypt / argon2), не в открытом виде.
- **Обработка ошибок:** внятные сообщения, если файл не существует / провайдер вернул 4xx или 5xx / превышен таймаут / LLM вернул невалидный JSON / превышен лимит минут. Не «голый stacktrace».
- **Таймауты и ретраи:** сетевые вызовы должны иметь таймаут и хотя бы 1 ретрай на 5xx/timeout.
- **Тесты** (`pytest` или `vitest`):
 - хотя бы один тест на «happy path» с **замоканными** API (без реальных вызовов);

- тесты на: невалидный JSON от LLM, ошибка сети у транскрипции, отсутствие файла;
- тест на лимит: пользователь израсходовал 30 минут — следующий запрос отклоняется;
- тест на изоляцию пользователей: пользователь А не видит задачи пользователя В;
- один тест на парсинг/валидацию структуры отчёта;
- **тесты должны проходить без интернета и без ключей.**
- **Git:** работа в репозитории GitHub, минимум 5 осмысленных коммитов с понятными сообщениями. Один большой коммит «final» = минус.

4. Бонусы (не обязательно)

- Абстракция провайдеров (интерфейс `TranscriptionProvider` / `LLMProvider`) — чтобы можно было поменять Deepgram на AssemblyAI правкой одной строки.
- Подсчёт стоимости запроса (длительность × тариф провайдера).
- Прогресс-бар и логи через `rich` / `chalk`.
- GitHub Actions: запуск тестов на push.
- Streaming-режим транскрипции, если провайдер поддерживает.
- Docker Compose для локального запуска одной командой.

5. Правила работы с AI-ассистентом

Использовать Claude, Cursor, ChatGPT, Copilot — **можно и нужно**. Это не считается «списыванием». Мы оцениваем не то, набил ли ты код руками, а то, насколько осознанно ты это делал.

Что мы хотим увидеть в README в разделе «Как я работал с AI»:

- какие задачи делегировал ассистенту, какие — нет;
- один пример, где ассистент **ошибся** или предложил плохое решение, и как ты это поймал;
- что менял руками поверх сгенерированного кода и почему.

Пустой раздел или «всё писал сам без AI» — это хуже, чем честный рассказ. Мы всё равно увидим по стилю.

6. Что мы будем оценивать

Критерий	Вес	На что смотрим
----------	-----	----------------

API-интеграция	25%	Таймауты, ретраи, обработка ошибок провайдеров, секреты через env, валидация ответов.
Тестирование	25%	Моки, edge cases, тесты не зависят от сети, осмысленные assertions (не <code>assert True</code>).
Авторизация и лимиты	15%	Хеширование паролей, изоляция данных между пользователями, корректный учёт минут.
Качество кода	15%	Структура, имена, нет мёртвого кода и AI-галлюцинаций («каркас на будущее»), нет лишних абстракций.
Работа с AI	10%	Раздел в README, осознанность, способность поймать неправоту ассистента.
Git и README	10%	Осмысленные коммиты, README, по которому реально можно запустить.
Бонусы	+5%	См. секцию 4.

7. Что сдавать

1. Ссылка на публичный (или приватный с доступом) GitHub-репозиторий.
2. В README — раздел «Демо»: либо короткий скринкаст / Loom (≤ 2 мин), либо текстовый лог последовательности запросов (curl / httpie) на реальном файле.
3. Финальное сообщение: ссылка + 3–5 предложений «что бы я улучшил, если бы было ещё 4 часа».

8. Дедлайн и формат связи

- На задачу — 5 календарных дней с момента получения.
- Вопросы можно и нужно задавать — но сначала попробуй разобраться сам и в формулировке вопроса покажи, что уже проверил.
- Если что-то непонятно или неоднозначно — **прими решение сам** и опиши его в README. «В задании не было сказано» — не аргумент.

Контакты

Юрий

CEO / Founder, LoyaBonus.app

+7 701 532 34 74

В случае заинтересованности результатом — пишите на WhatsApp по этому номеру.

Удачи. Не геройствуй: лучше маленькое работающее решение с честными тестами, чем большое и сломанное.